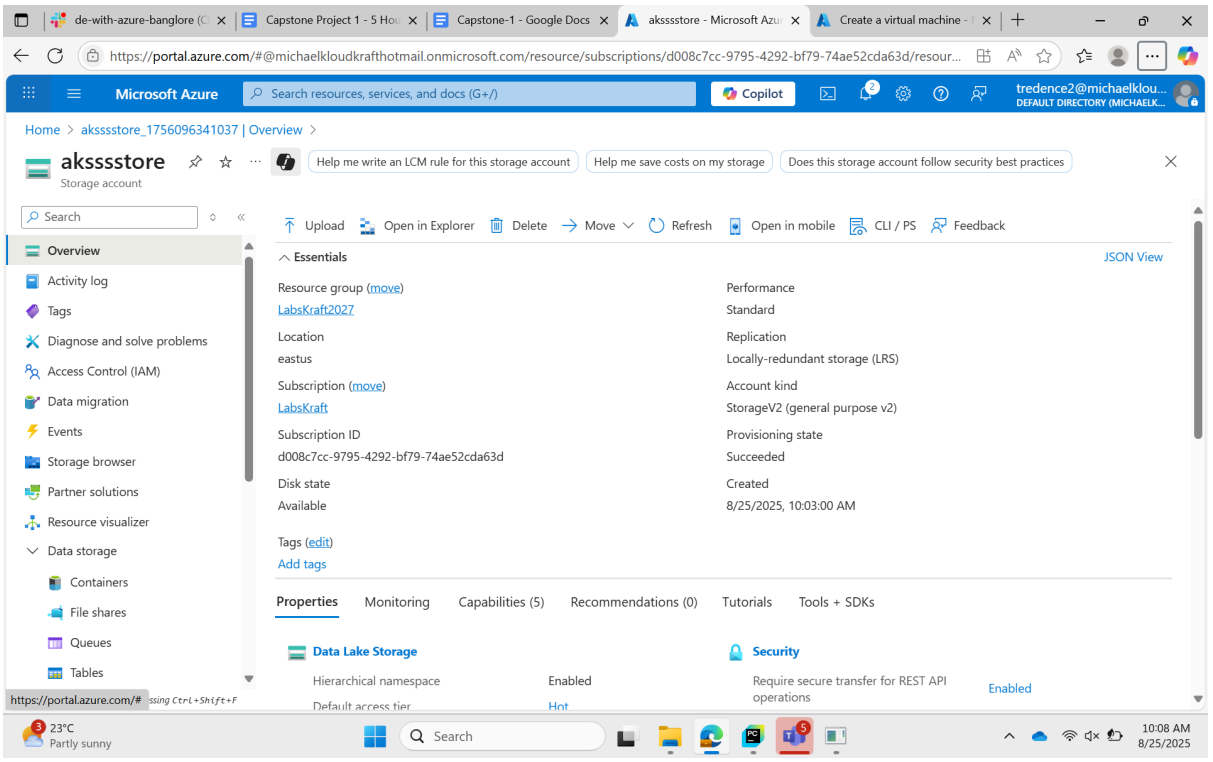
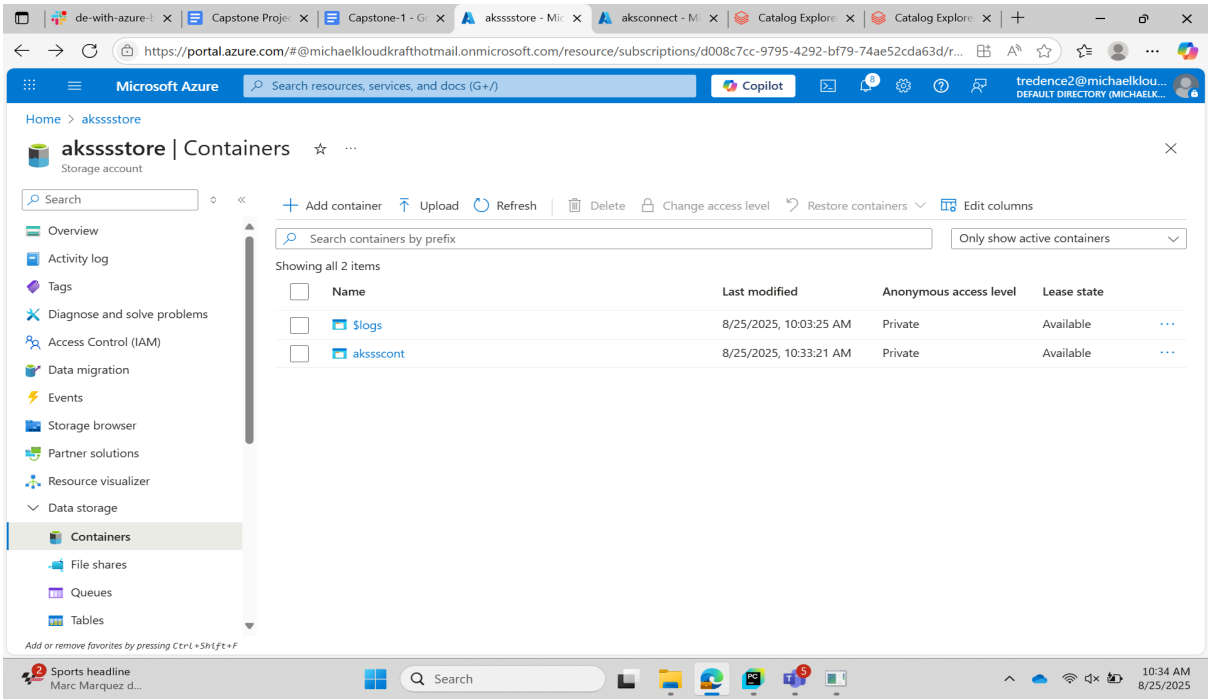
Akshita Bhamidimarri
Capstone Project - 1

IOT-Enabled Retail Analytics on Azure Databricks

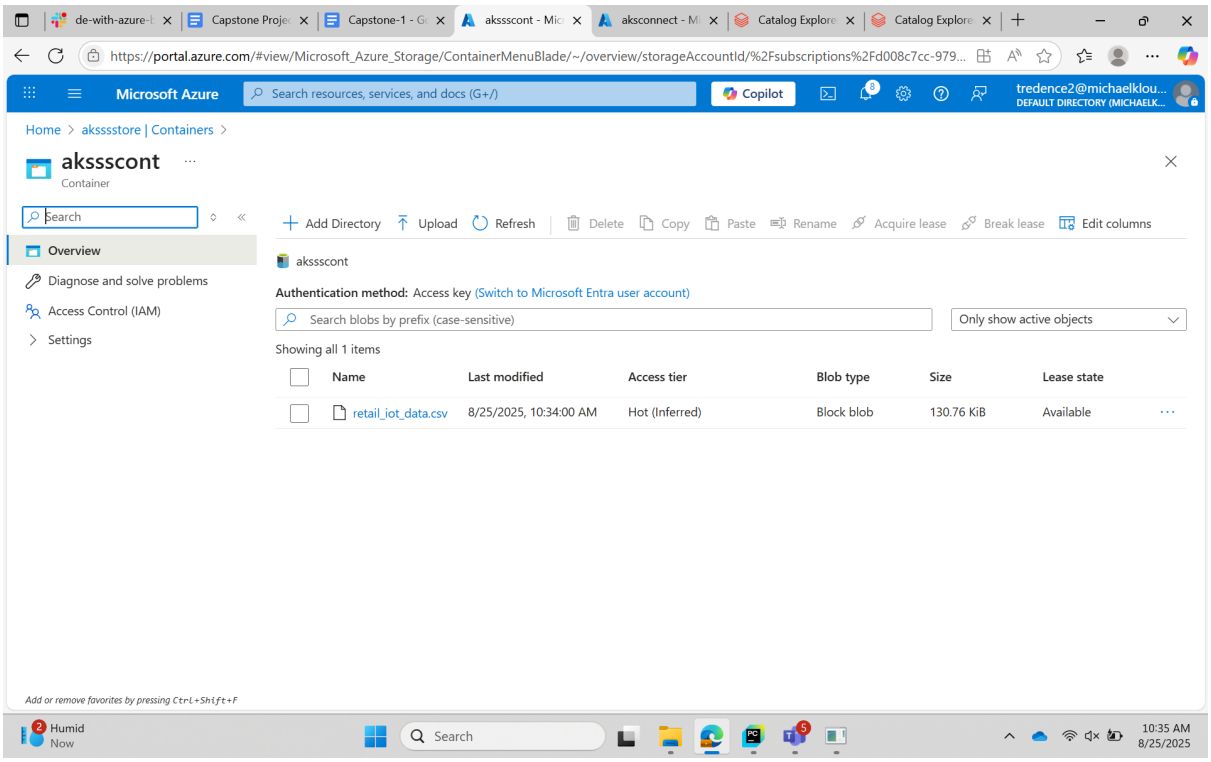
1) Creating a storage account



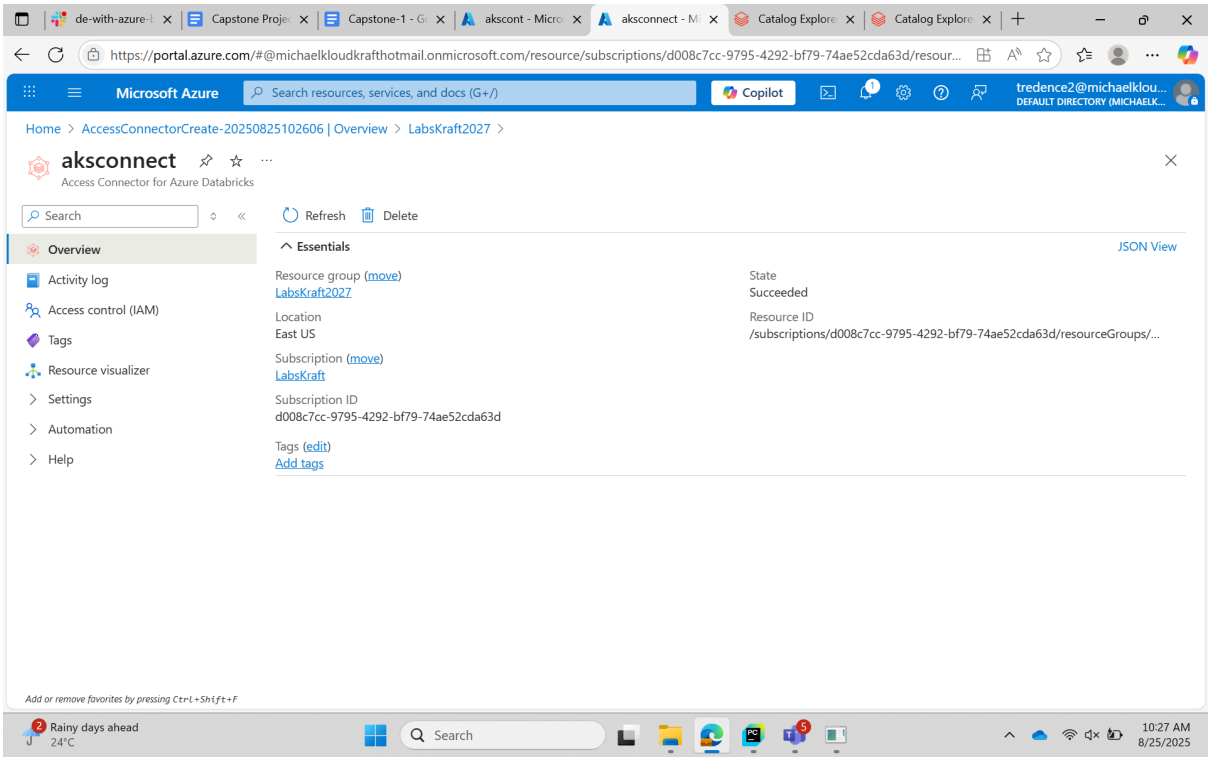
2) Creating a container inside the storage account



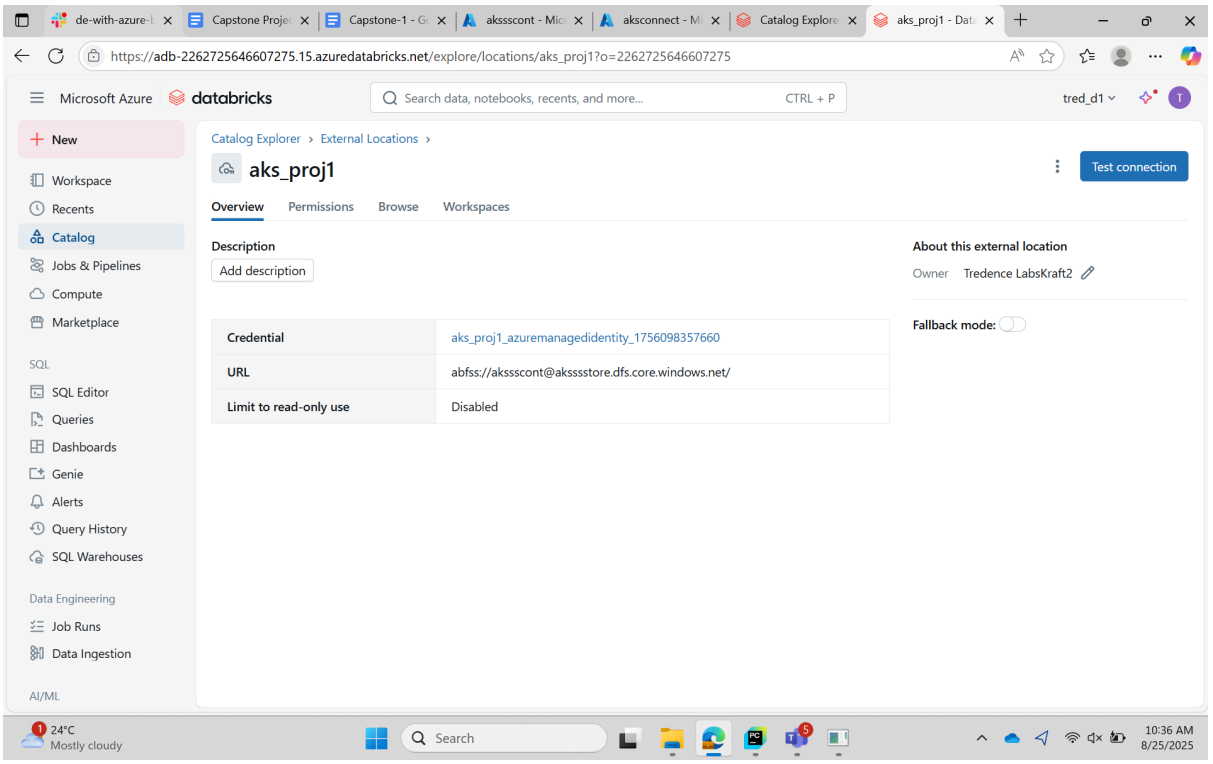
3) Adding the csv file inside the storage account

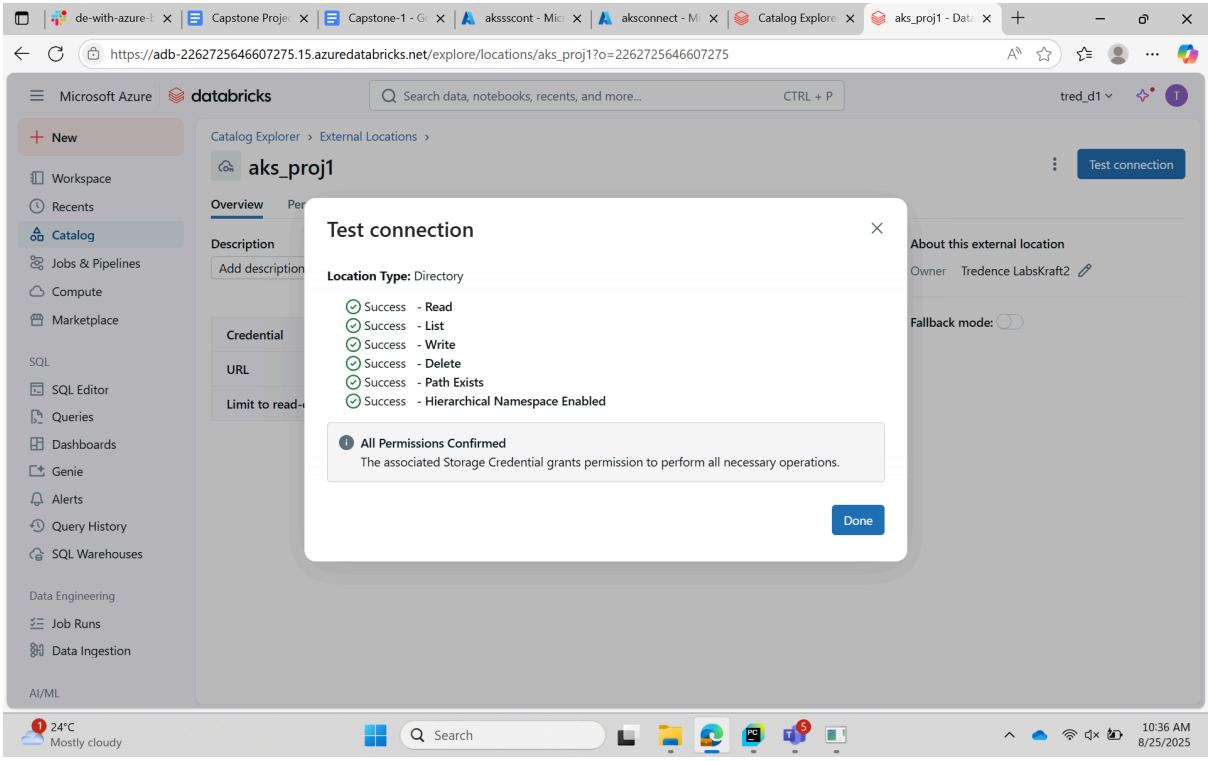


4) Creating an access connector for databricks

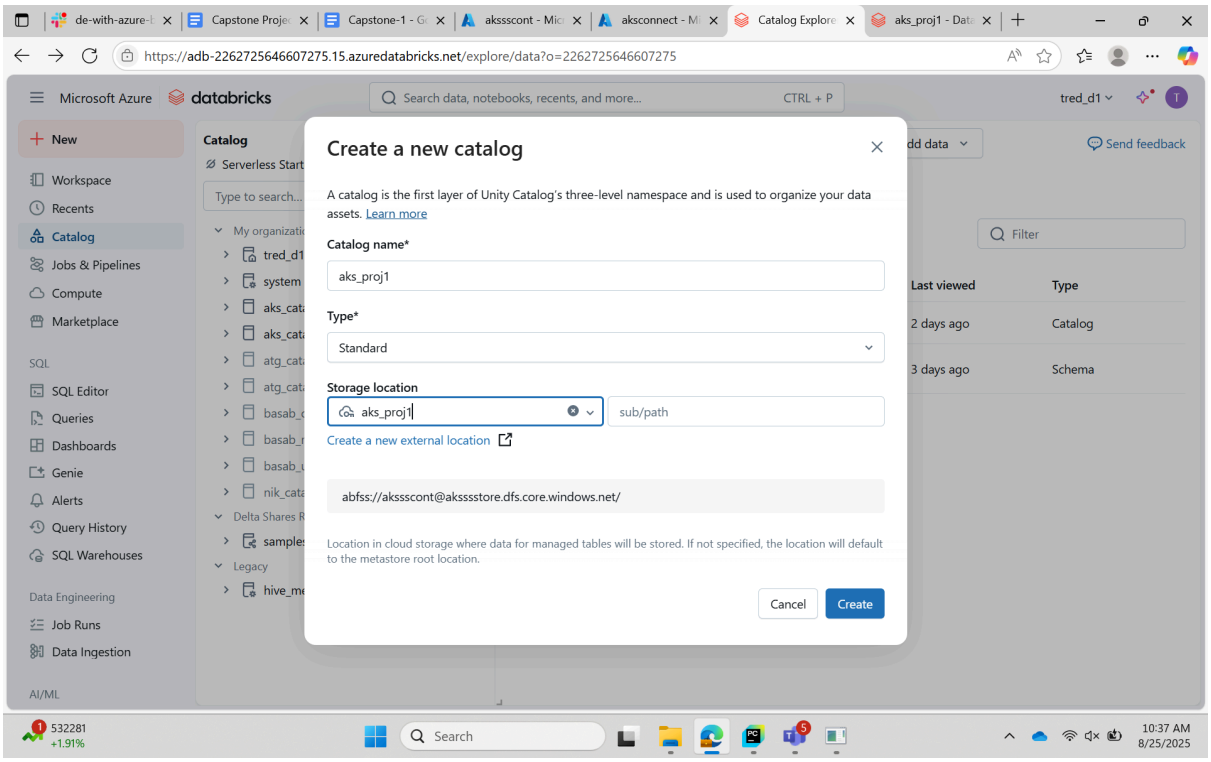


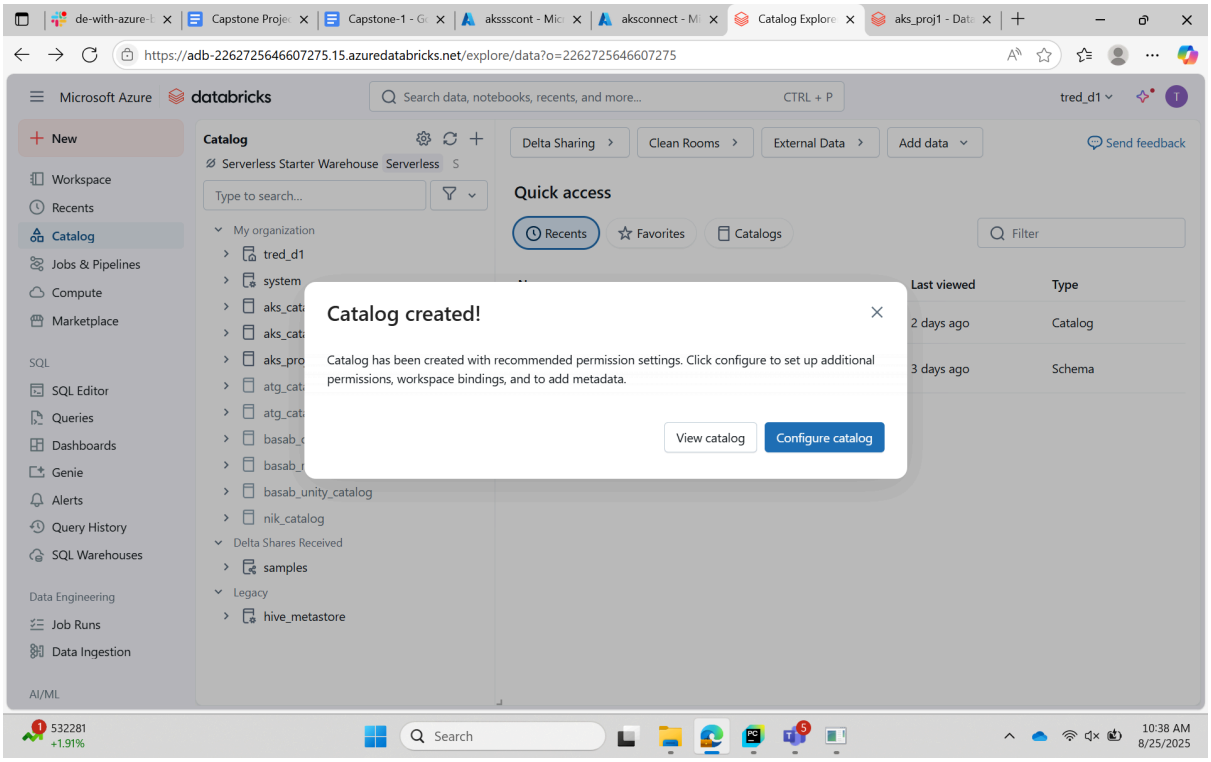
5) Creating the external location for unity catalog



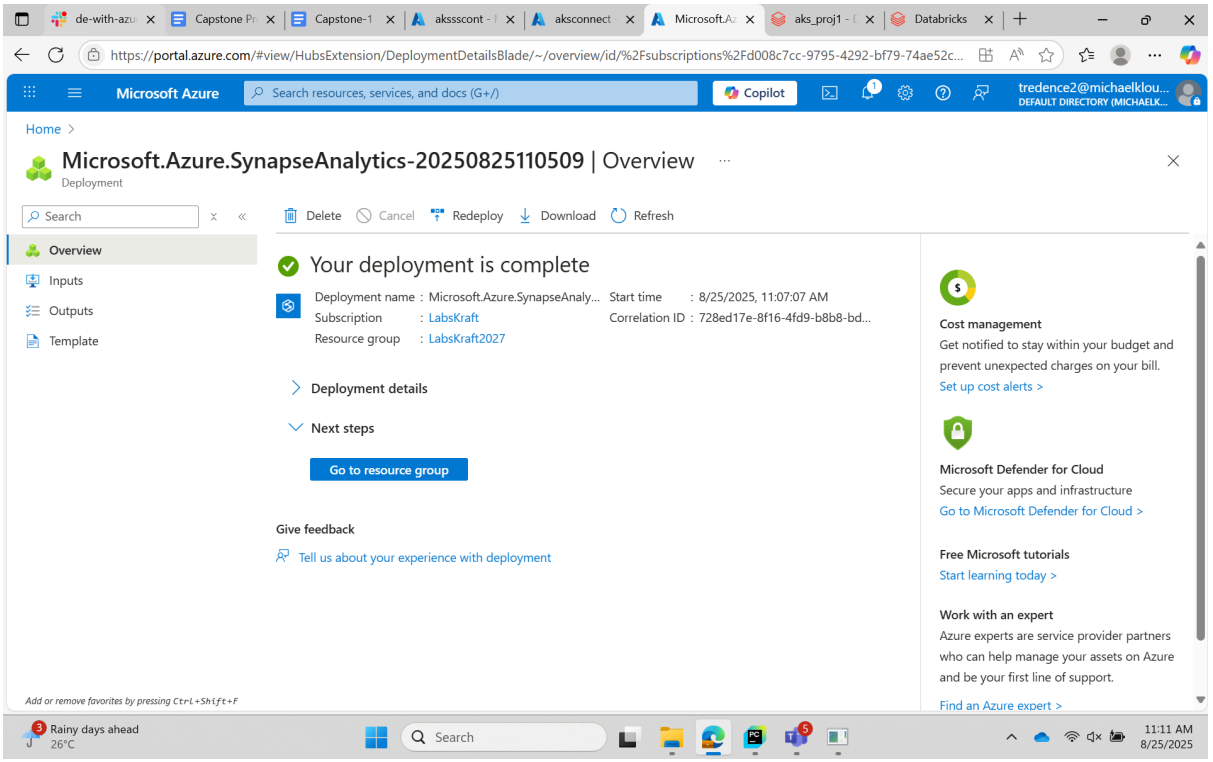


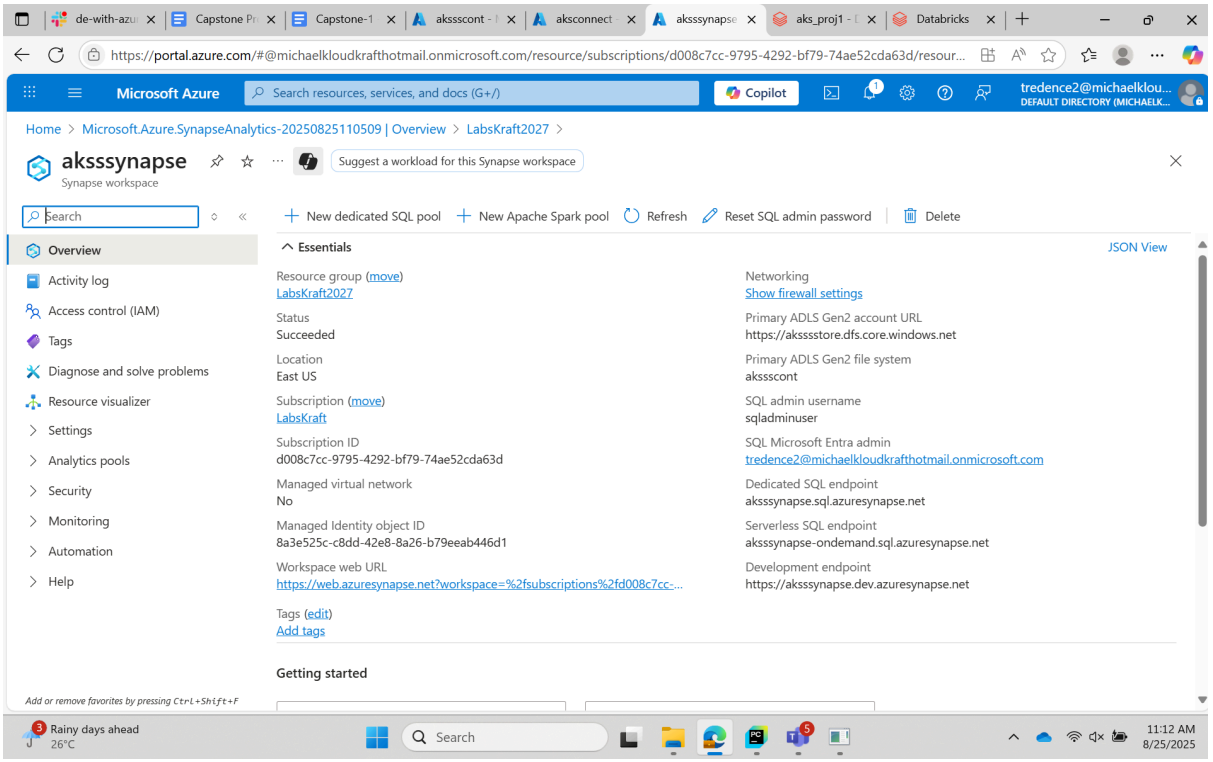
6) Creating the unity catalog using the previously created external location

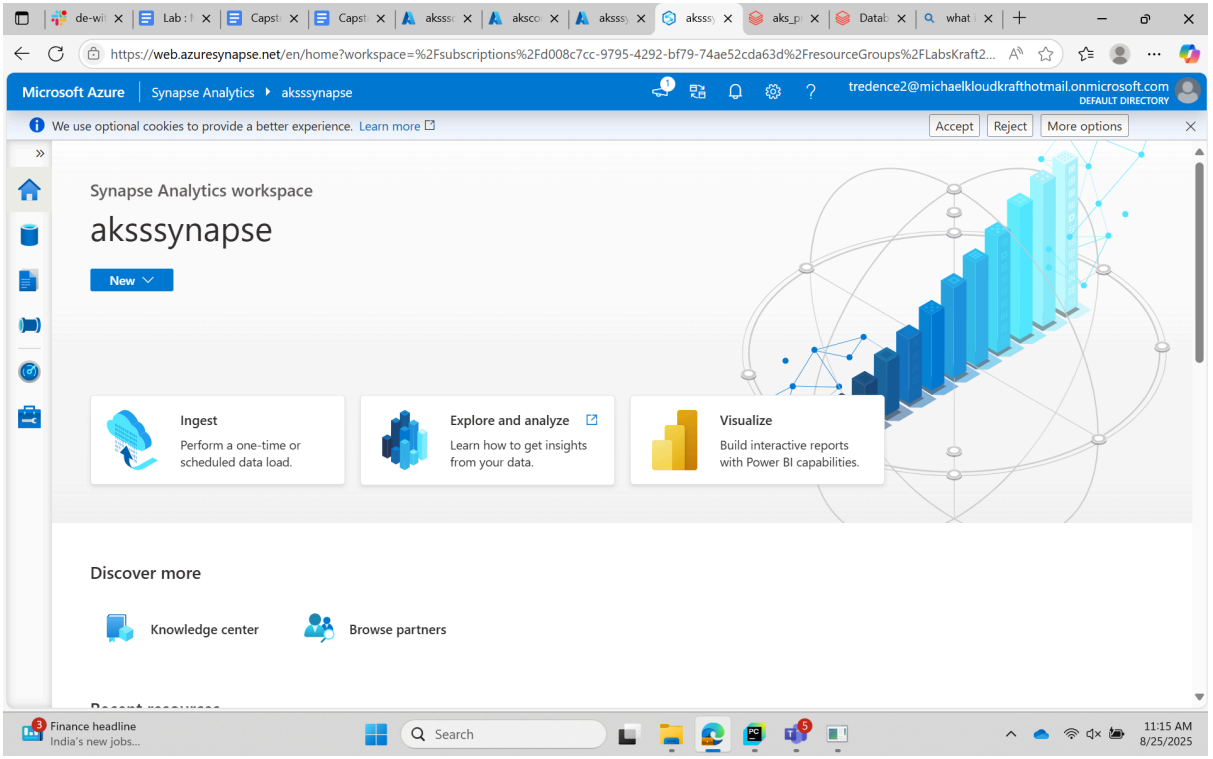




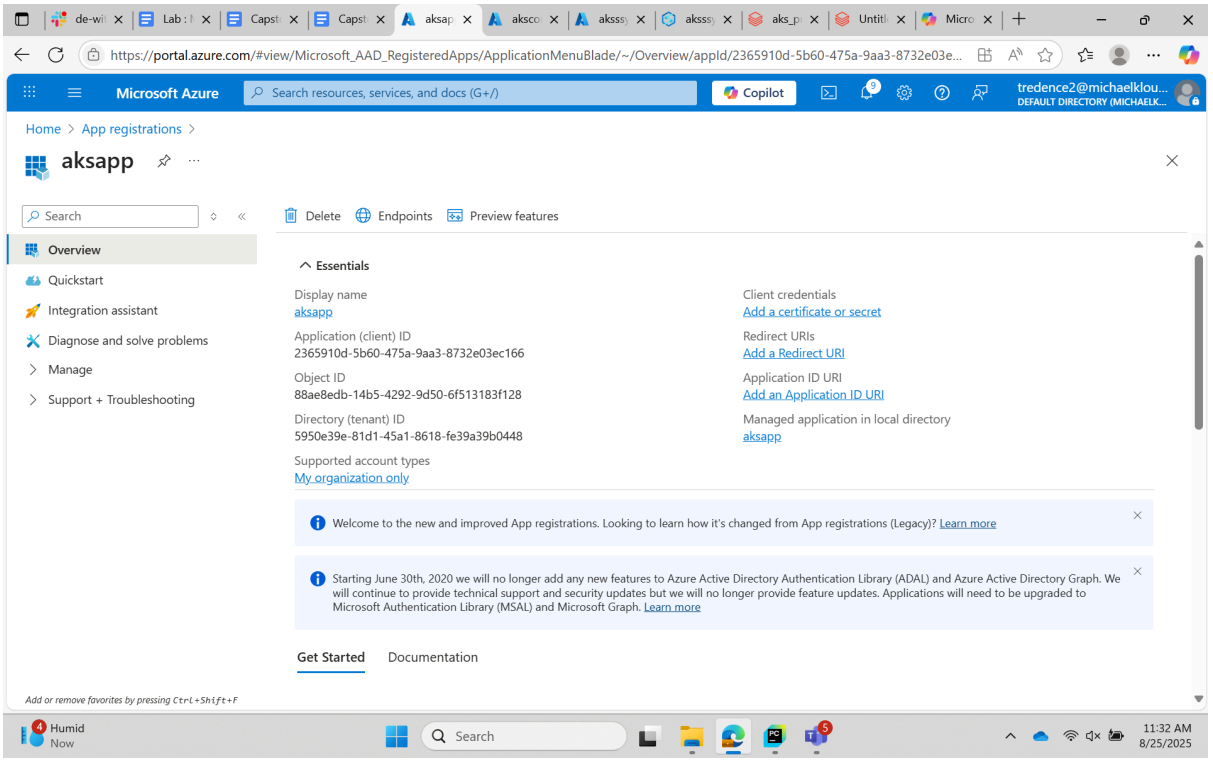
7) Deploying a new azure synapse analytics



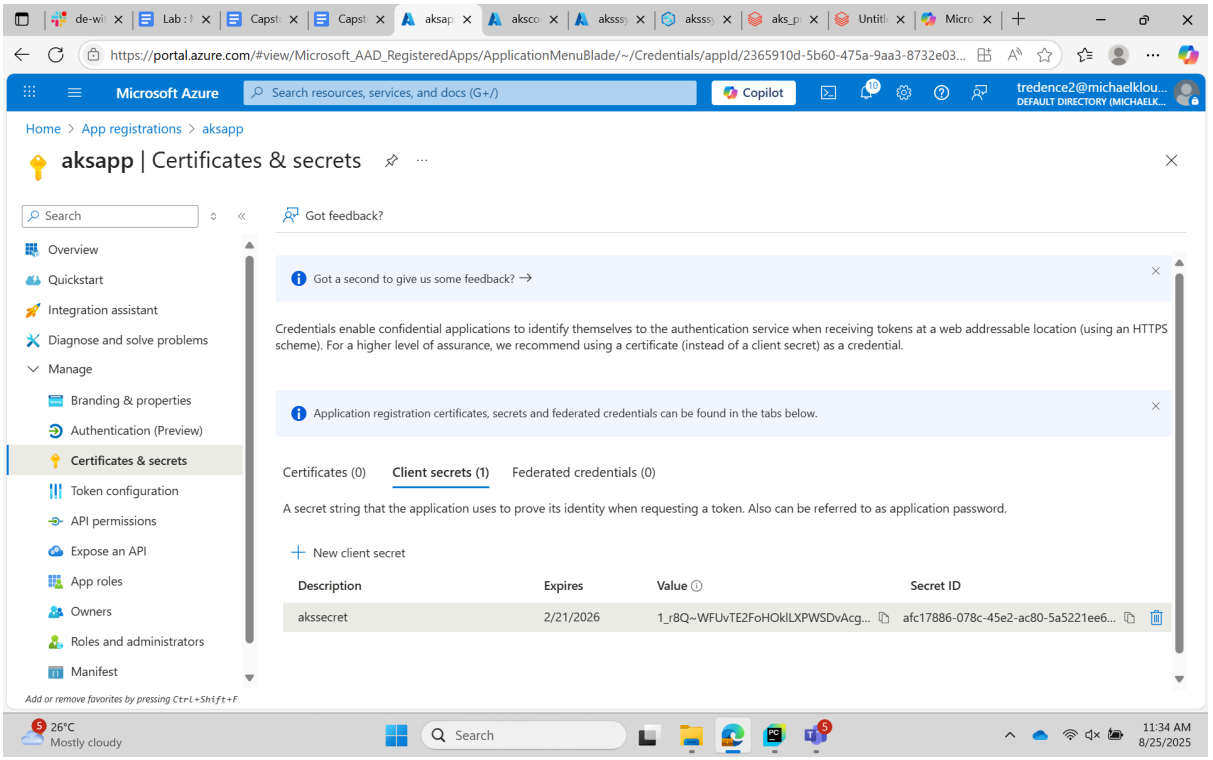




8) Creating a new app registration



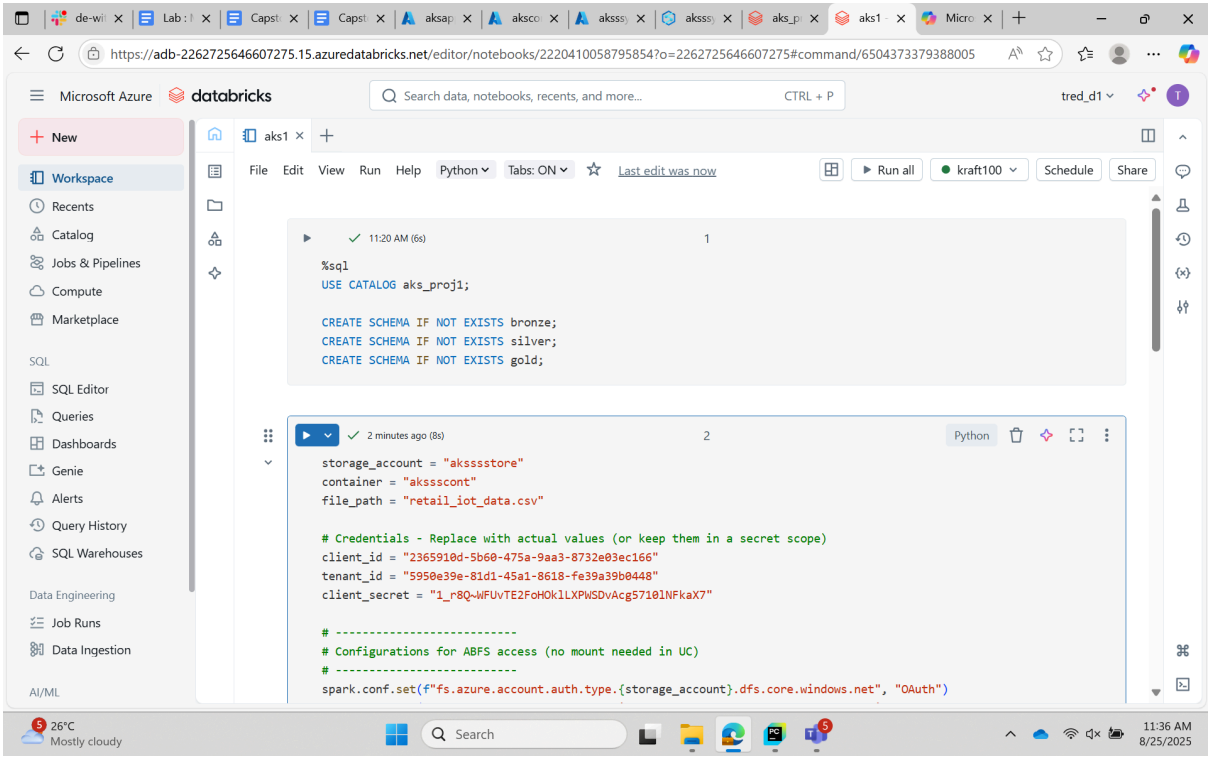
9) Generating a new secret value



10) Creating three schemas in the medallion architecture

Code for schema creation -

```
%sql
USE CATALOG aks_proj1;

CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;
```

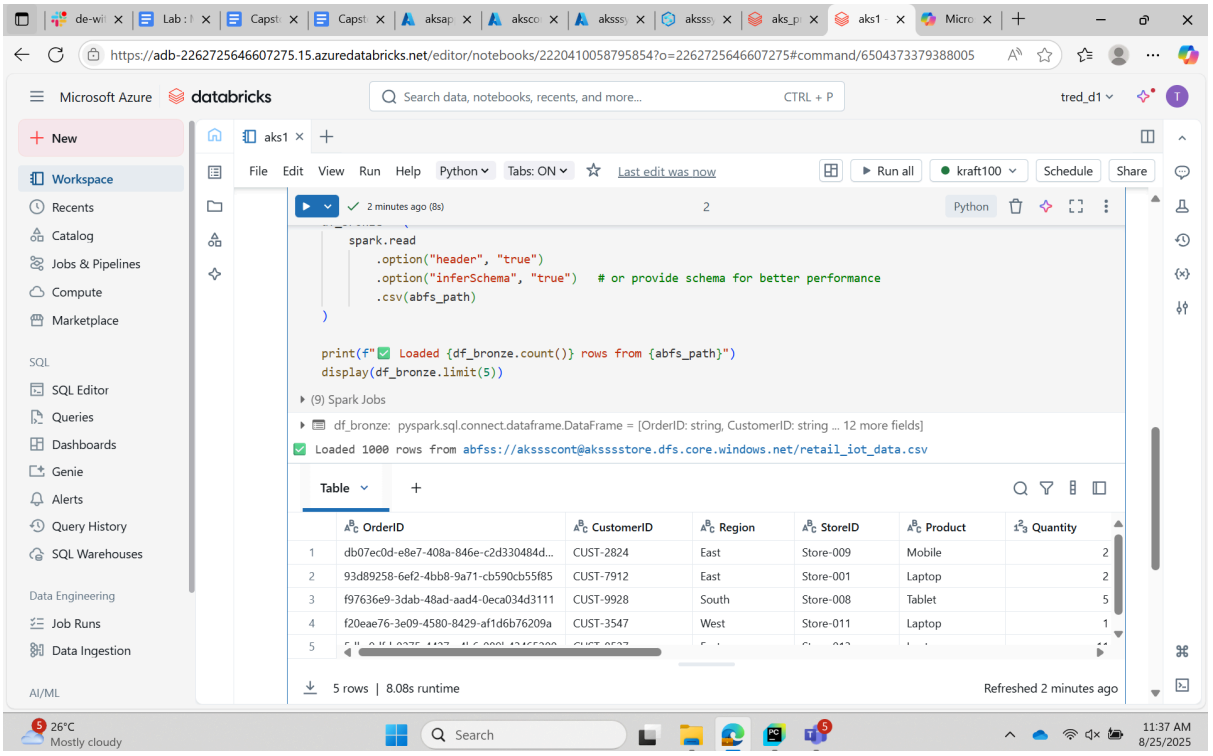


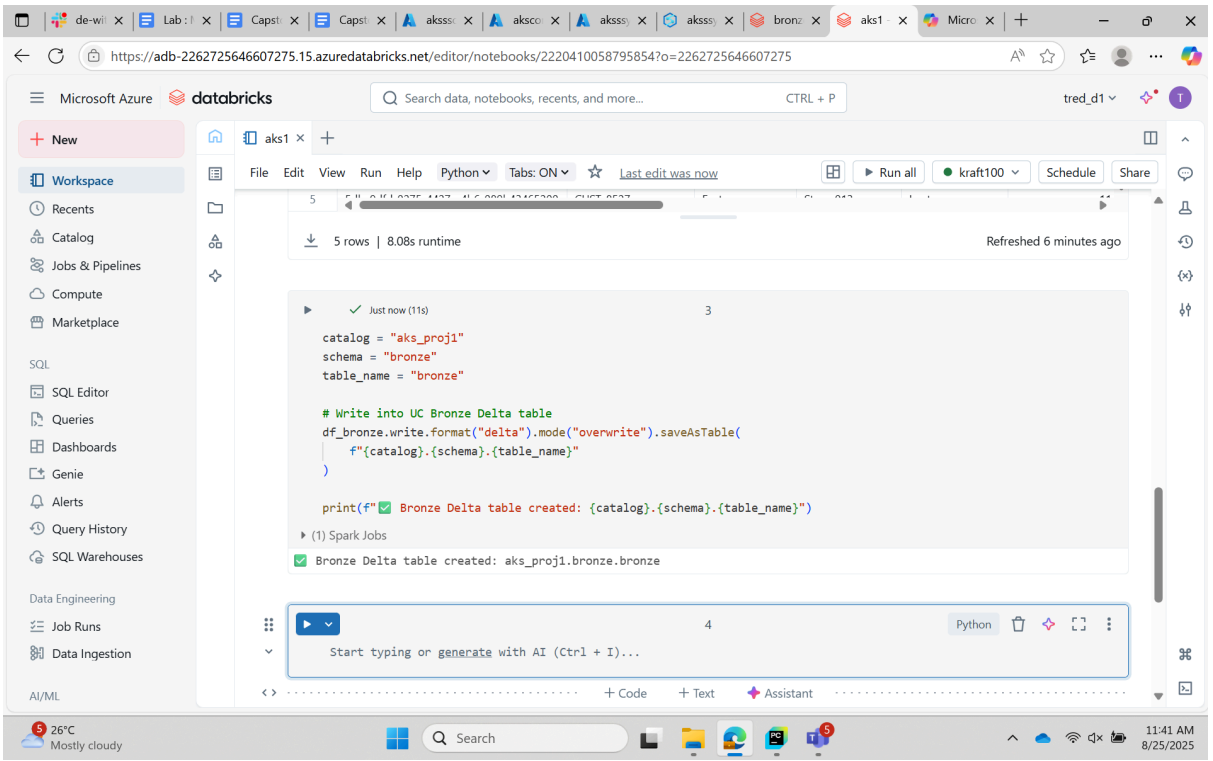
Code for bronze layer:

```
storage_account = "akssssstore"
container = "aksssconT"
file_path = "retail_iot_data.csv"

# Credentials - Replace with actual values (or keep them in a secret scope)
client_id = "2365910d-5b60-475a-9aa3-8732e03ec166"
tenant_id = "5950e39e-81d1-45a1-8618-fe39a39b0448"
client_secret = "1_r8Q~WFUvTE2FoHOklLXPWSDvAcg5710lNFkaX7"

# ---------------------------
# Configurations for ABFS access (no mount needed in UC)
# ---------------------------
spark.conf.set(f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net", "OAuth")
spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",

"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net", client_id)
```

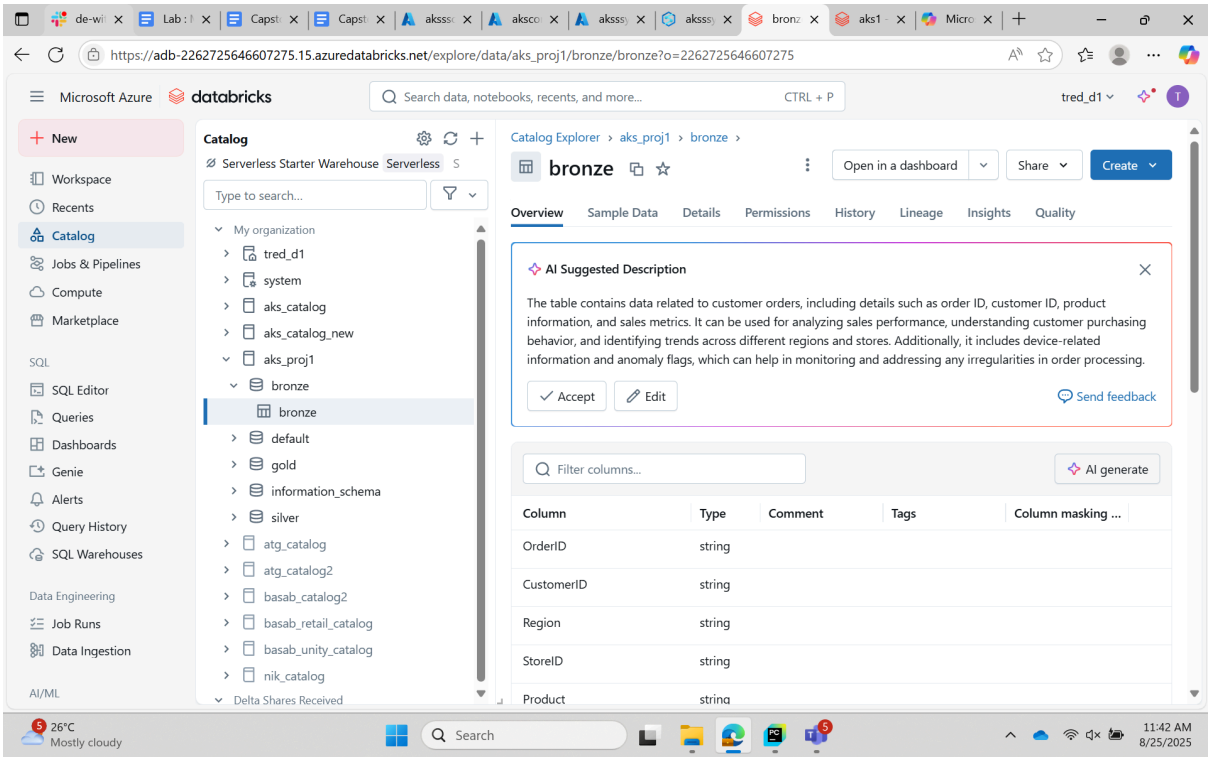
```
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.
core.windows.net", client_secret)
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account}.df
s.core.windows.net",
               f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")

# ----------------------------
# Build ABFS path directly (no /mnt needed)
# ----------------------------
abfs_path =
f"abfss://{container}@{storage_account}.dfs.core.windows.net/{file_path}"

# ----------------------------
# Try reading the file to verify access
# ----------------------------
# Read raw file from ADLS
df_bronze = (
    spark.read
        .option("header", "true")
        .option("inferSchema", "true")   # or provide schema for better
performance
        .csv(abfs_path)
)

print(f"✅ Loaded {df_bronze.count()} rows from {abfs_path}")
display(df_bronze.limit(5))
```

```
catalog = "aks_proj1"
schema = "bronze"
table_name = "bronze"

# Write into UC Bronze Delta table
df_bronze.write.format("delta").mode("overwrite").saveAsTable(
    f"{catalog}.{schema}.{table_name}"
)

print(f"✅ Bronze Delta table created: {catalog}.{schema}.{table_name}")
```

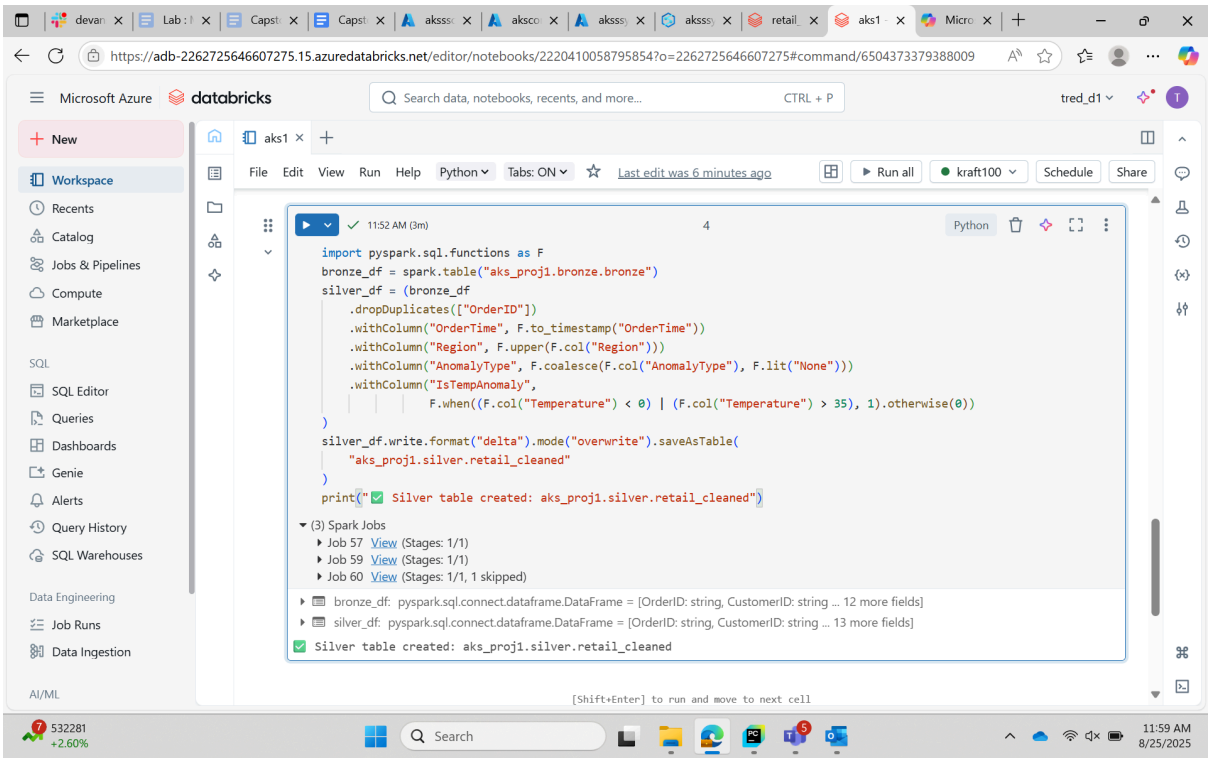

https://adb-2262725646607275.15.azuredatabricks.net/editor/notebooks/2220410058795854?o=2262725646607275#command/6504373379388005
Microsoft Azure
databricks
Search data, notebooks, recents, and more... CTRL + P
tred_d1
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
aks1
File Edit View Run Help Python Tabs: ON Last edit was now
Run all kraft100 Schedule Share
2 minutes ago (8s)
2
Python
spark.read
.option("header", "true")
.option("inferSchema", "true") # or provide schema for better performance
.csv(abfs_path)
print(f"✅ Loaded {df_bronze.count()} rows from {abfs_path}")
display(df_bronze.limit(5))
(9) Spark Jobs
df_bronze: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]
Loaded 1000 rows from abfss://aksscont@akssstore.dfs.core.windows.net/retail_iot_data.csv
Table
OrderID CustomerID Region StoreID Product Quantity
1 db07ec0d-e8e7-408a-846e-c2d330484d... CUST-2824 East Store-009 Mobile 2
2 93d89258-6ef2-4bb8-9a71-cb590cb55f85 CUST-7912 East Store-001 Laptop 2
3 f97636e9-3dab-48ad-aad4-0eca034d3111 CUST-9928 South Store-008 Tablet 5
4 f20eae76-3e09-4580-8429-af1d6b76209a CUST-3547 West Store-011 Laptop 1
5
5 rows | 8.08s runtime
Refreshed 2 minutes ago
26°C
Mostly cloudy
Search
11:37 AM
8/25/2025


https://adb-2262725646607275.15.azuredatabricks.net/editor/notebooks/2220410058795854?o=2262725646607275
Microsoft Azure
databricks
Search data, notebooks, recents, and more... CTRL + P
tred_d1
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
aks1
File Edit View Run Help Python Tabs: ON Last edit was now
Run all kraft100 Schedule Share
5 rows | 8.08s runtime
Refreshed 6 minutes ago
Just now (11s)
3
catalog = "aks_proj1"
schema = "bronze"
table_name = "bronze"
# Write into UC Bronze Delta table
df_bronze.write.format("delta").mode("overwrite").saveAsTable(
f"{catalog}.{schema}.{table_name}"
)
print(f"✅ Bronze Delta table created: {catalog}.{schema}.{table_name}")
(1) Spark Jobs
Bronze Delta table created: aks_proj1.bronze.bronze
4
Python
Start typing or generate with AI (Ctrl + I)...
+ Code + Text Assistant
26°C
Mostly cloudy
Search
11:41 AM
8/25/2025

11) Bronze table that we just created in the schema with the appropriate columns

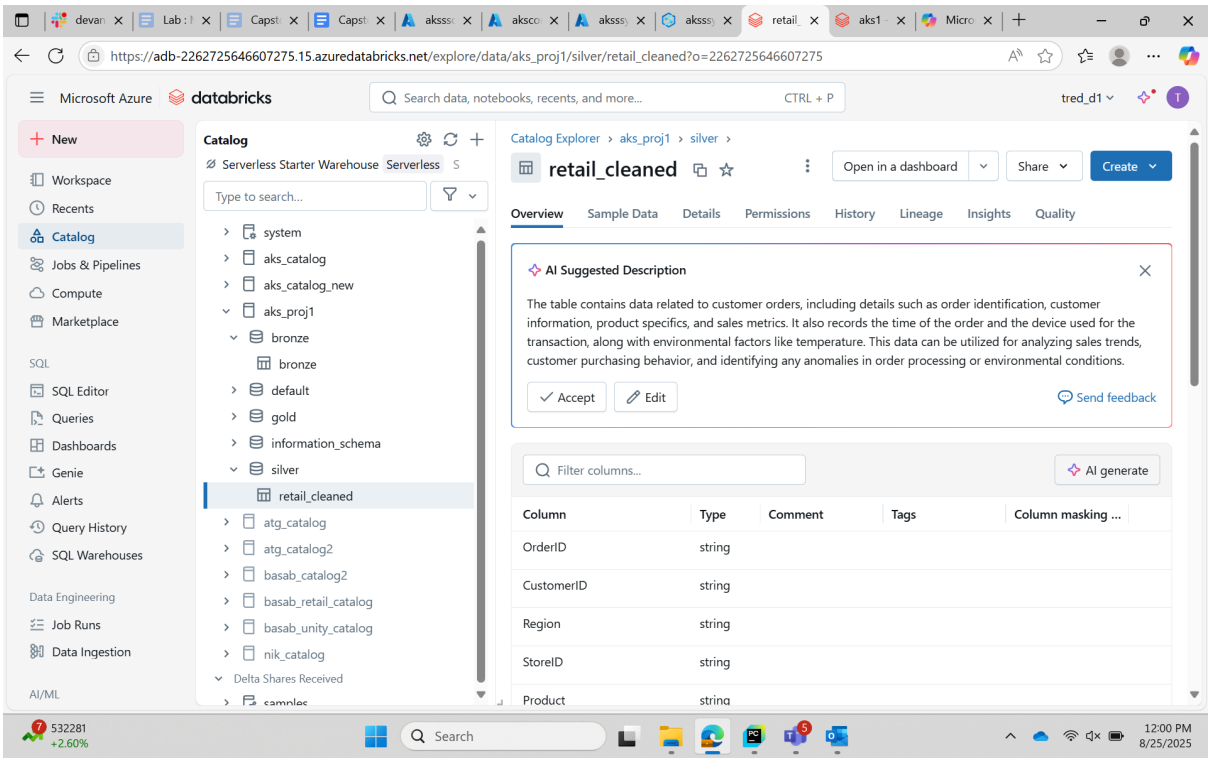


12) Creating the silver layer



Code for silver layer:

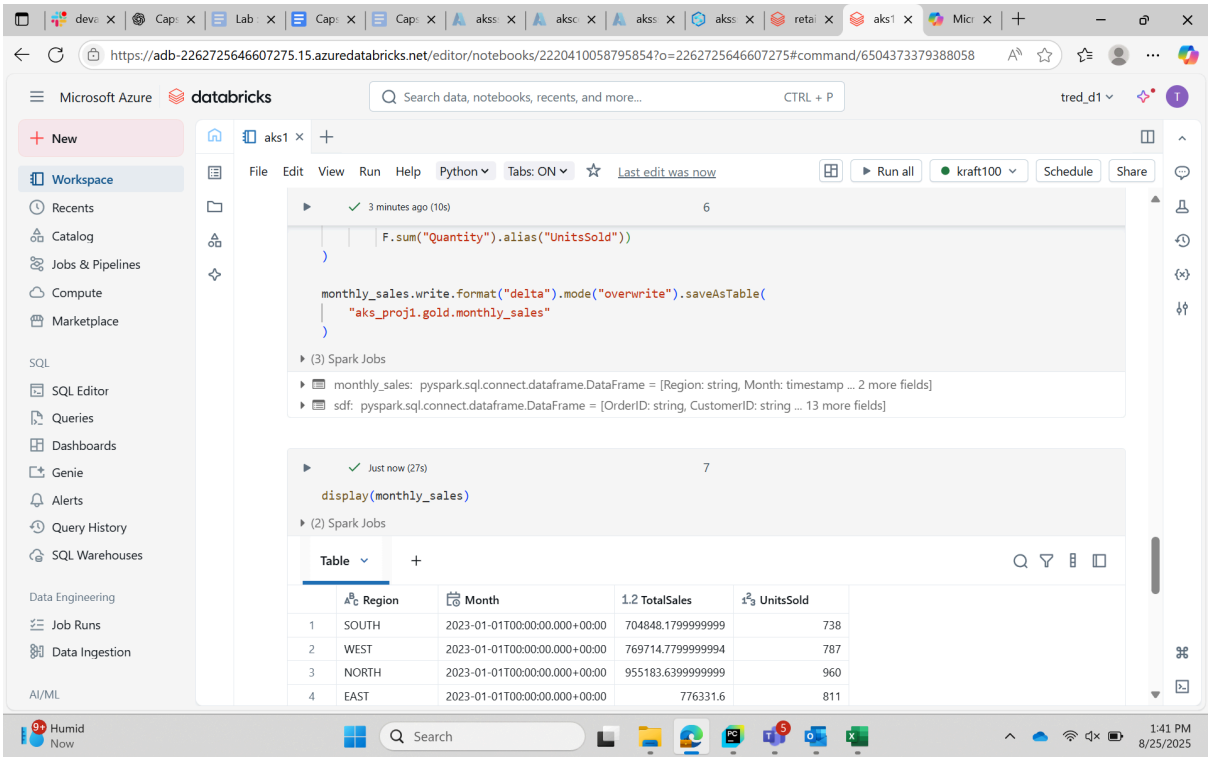
```
import pyspark.sql.functions as F
bronze_df = spark.table("aks_proj1.bronze.bronze")
silver_df = (bronze_df
    .dropDuplicates(["OrderID"])
    .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    .withColumn("Region", F.upper(F.col("Region")))
    .withColumn("AnomalyType", F.coalesce(F.col("AnomalyType"),
F.lit("None")))
    .withColumn("IsTempAnomaly",
                F.when((F.col("Temperature") < 0) | (F.col("Temperature") >
35), 1).otherwise(0))
)
silver_df.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.silver.retail_cleaned"
)
print("✅ Silver table created: aks_proj1.silver.retail_cleaned")
```



13) Creating different KPI's for the gold layer

Code for KPI 1 (daily/montly sales per region):

```
#gold layer kpi1 - dialy/monthly sales per region

import pyspark.sql.functions as F

sdf = spark.table("aks_proj1.silver.retail_cleaned")

monthly_sales = (sdf
    .groupBy("Region", F.date_trunc("month", "OrderTime").alias("Month"))
    .agg(F.sum("SalesAmount").alias("TotalSales"),
         F.sum("Quantity").alias("UnitsSold"))
)

monthly_sales.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.monthly_sales"
)
```



Code for KPI 2 (anomaly trend per store):

```
#gold layer kpi2 - anomaly trend per store

anomaly_trend = (sdf
    .groupBy("StoreID", F.to_date("OrderTime").alias("Day"))
    .agg(F.sum("AnomalyFlag").alias("SystemAnomalies"),
         F.sum("IsTempAnomaly").alias("TempAnomalies"),
         F.sum("SalesAmount").alias("Sales"))
)

anomaly_trend.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.anomaly_trend"
)
```

Code for KPI 3 (impact of divide failure):

```
#gold layer kpi3 - impact of device failures

import pyspark.sql.functions as F
sdf = spark.table("aks_proj1.silver.retail_cleaned")

impact = (sdf
    .withColumn("AnomalyDetected", (F.col("AnomalyFlag") +
F.col("IsTempAnomaly")) > 0)
    .groupBy("AnomalyDetected")
    .agg(F.sum("SalesAmount").alias("TotalSales"),
         F.countDistinct("OrderID").alias("Orders"))
)

impact.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.anomaly_impact"
)
```

Code for KPI 4 (stores into tier):

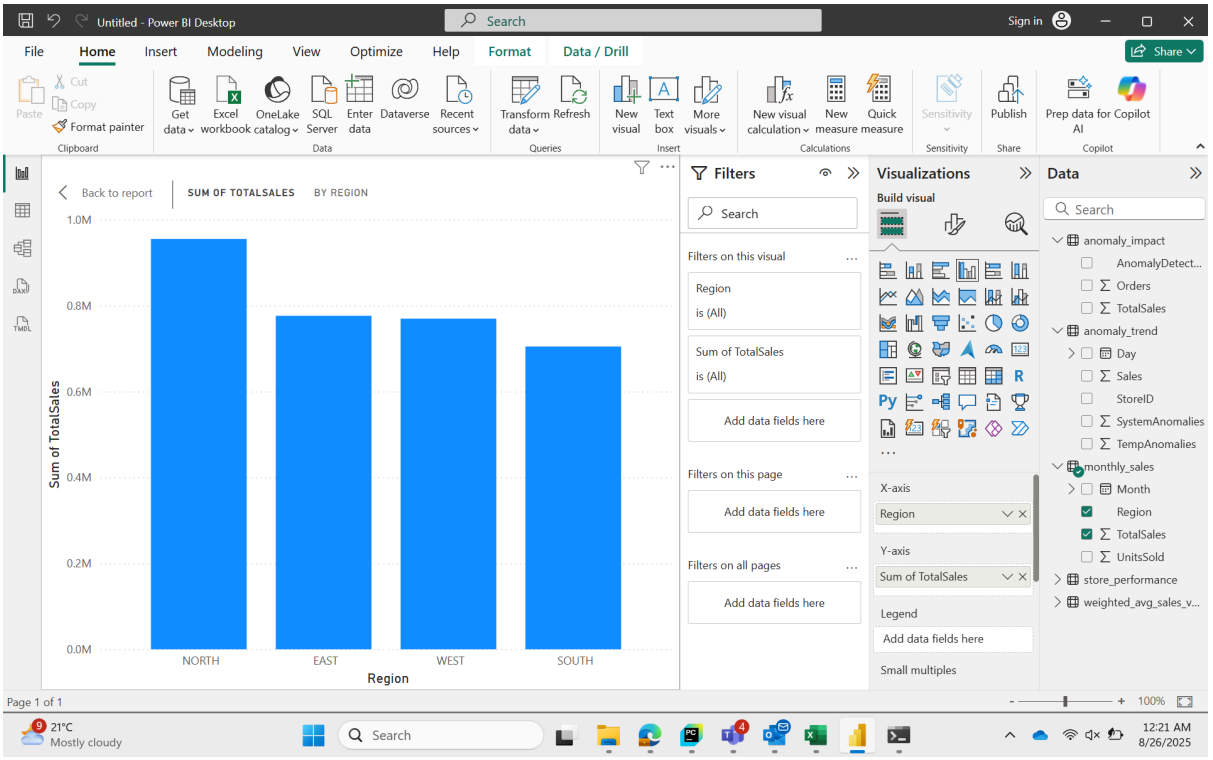
```
#gold layer kpi4 - store into tiers

store_perf = (sdf
    .groupBy("StoreID")
    .agg(F.sum("SalesAmount").alias("TotalSales"))
    .withColumn("Tier",
        F.when(F.col("TotalSales") > 50000, "High")
         .when(F.col("TotalSales") > 20000, "Medium")
         .otherwise("Low"))
)

store_perf.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.store_performance"
)
```

Code for KPI 5 (weighted average of sales vs anomalies):

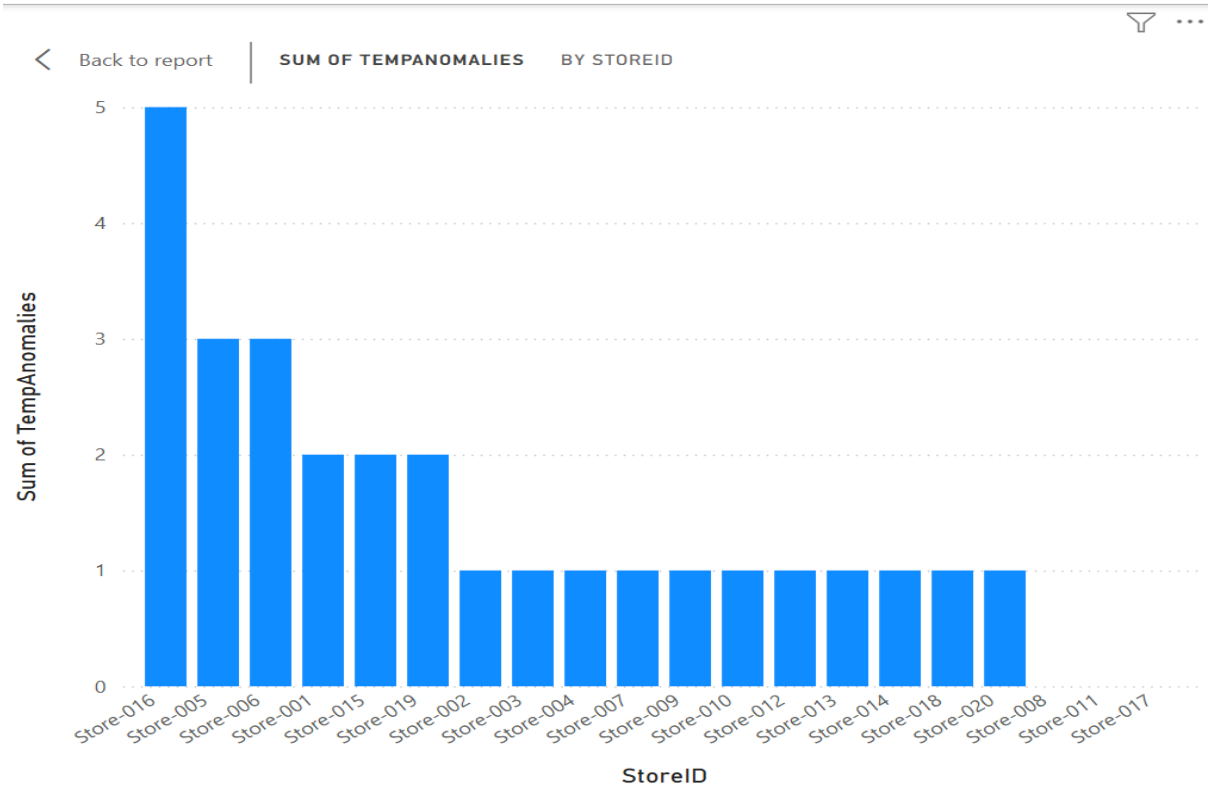
```
#gold layer kpi5 - weighted average of sales vs anomalies

store_sales_anoms = (sdf
    .groupBy("StoreID")
    .agg(F.sum("SalesAmount").alias("TotalSales"),
         F.sum("AnomalyFlag").alias("AnomalyCount"))
    .filter("AnomalyCount > 0")  # only consider stores with anomalies
)
# Compute weighted average: (Sales * AnomalyCount) / Sum(AnomalyCount)
weighted_avg = (store_sales_anoms
    .select((F.sum(F.col("TotalSales") * F.col("AnomalyCount")) /
             F.sum("AnomalyCount")).alias("WeightedAvgSales"))
)
weighted_avg.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.weighted_avg_sales_vs_anomalies"
)
```

14) Power BI dashboards

Total sales per region



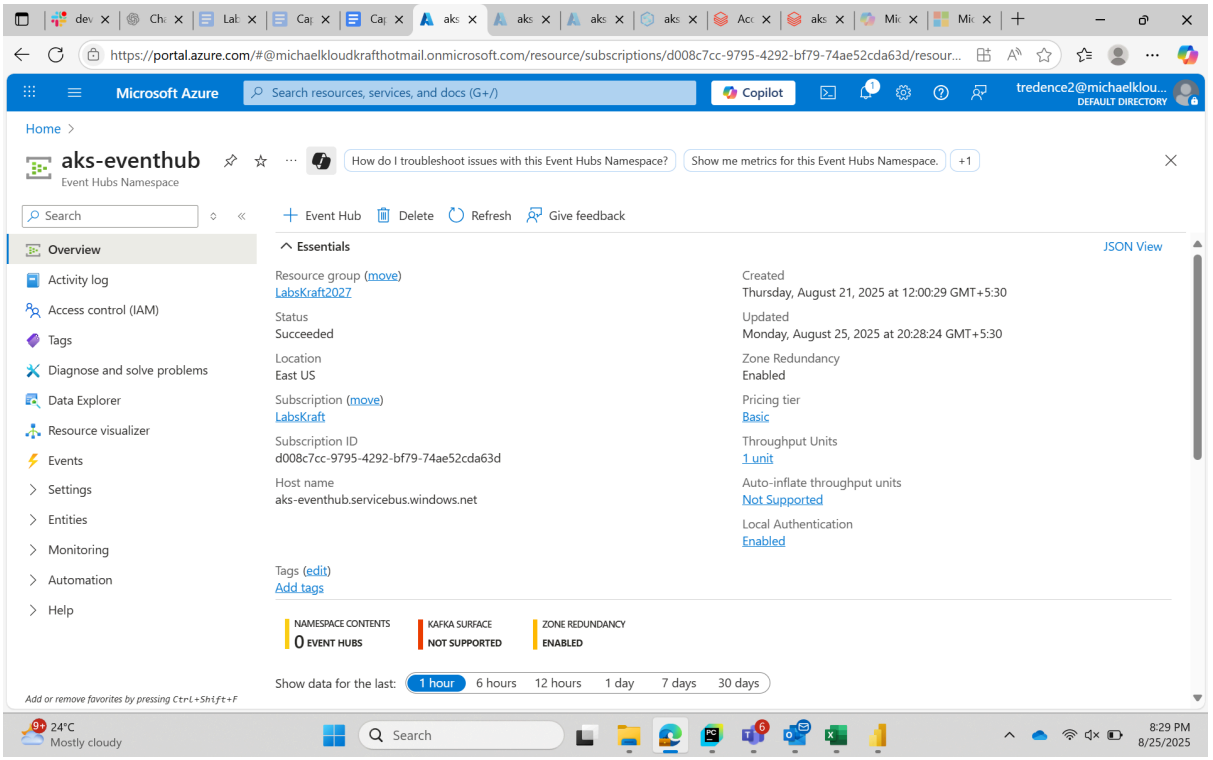
Sum of Anomalies per store ID



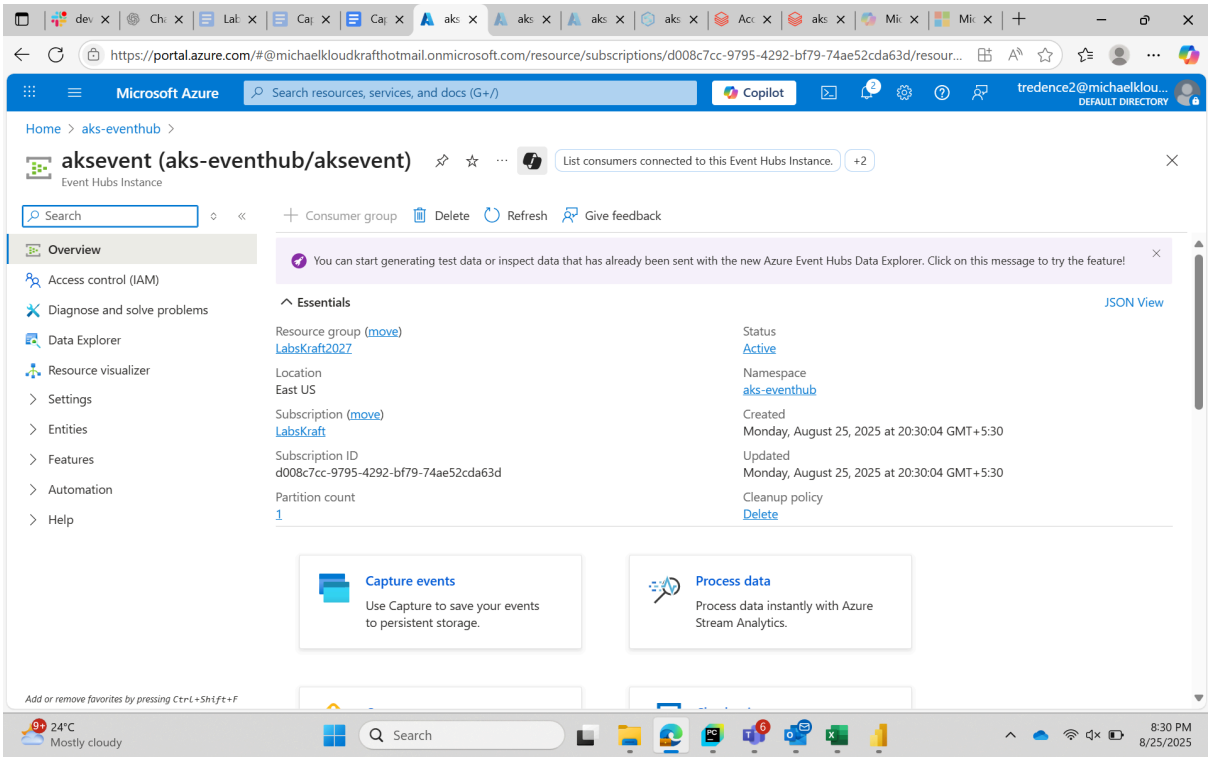
15) Optimization of the queries

```
%sql
OPTIMIZE aks_proj1.gold.monthly_sales ZORDER BY (Month);
%sql
OPTIMIZE aks_proj1.gold.anomaly_trend ZORDER BY (StoreID, Day);
%sql
OPTIMIZE aks_proj1.gold.anomaly_impact ZORDER BY (AnomalyDetected);
%sql
OPTIMIZE aks_proj1.gold.store_performance ZORDER BY (TotalSales);
%sql
OPTIMIZE aks_proj1.gold.weighted_avg_sales_vs_anomalies;
%sql
OPTIMIZE aks_proj1.gold.monthly_sales;
OPTIMIZE aks_proj1.gold.anomaly_trend;
OPTIMIZE aks_proj1.gold.anomaly_impact;
OPTIMIZE aks_proj1.gold.store_performance;
OPTIMIZE aks_proj1.gold.weighted_avg_sales_vs_anomalies;
```

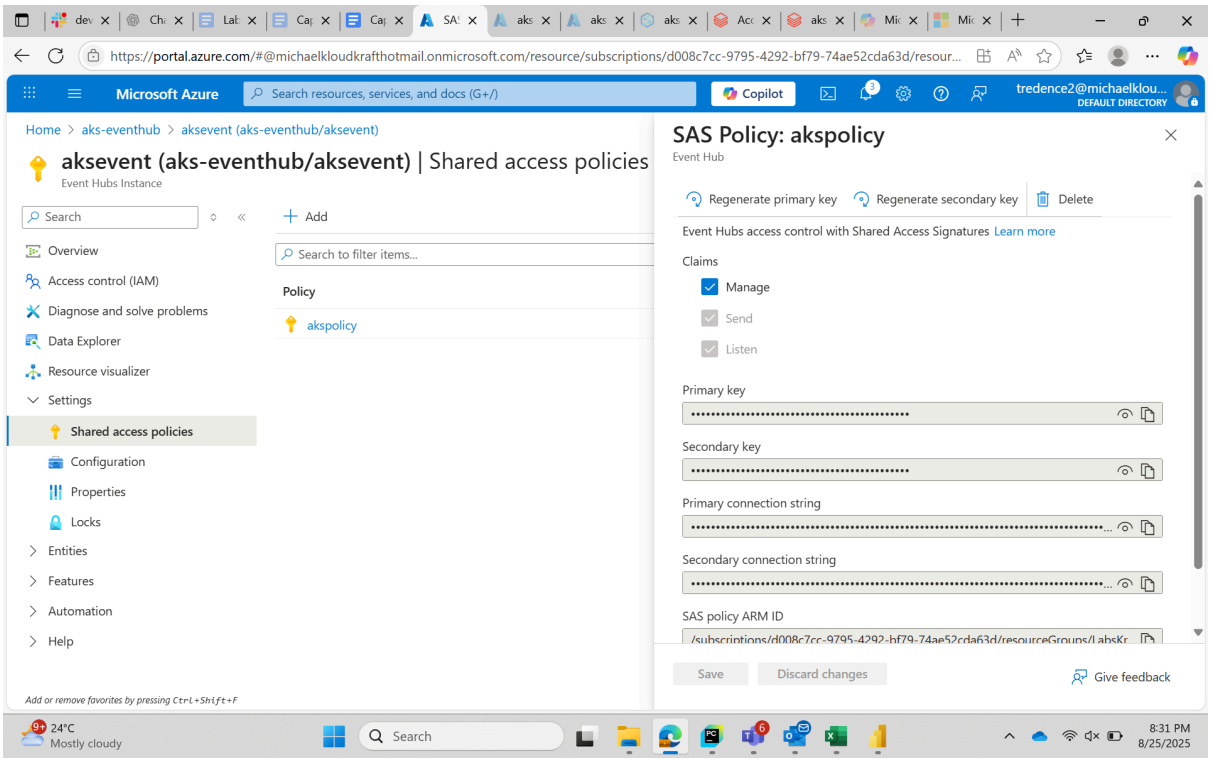
16) Using Event Hub as a Data Source



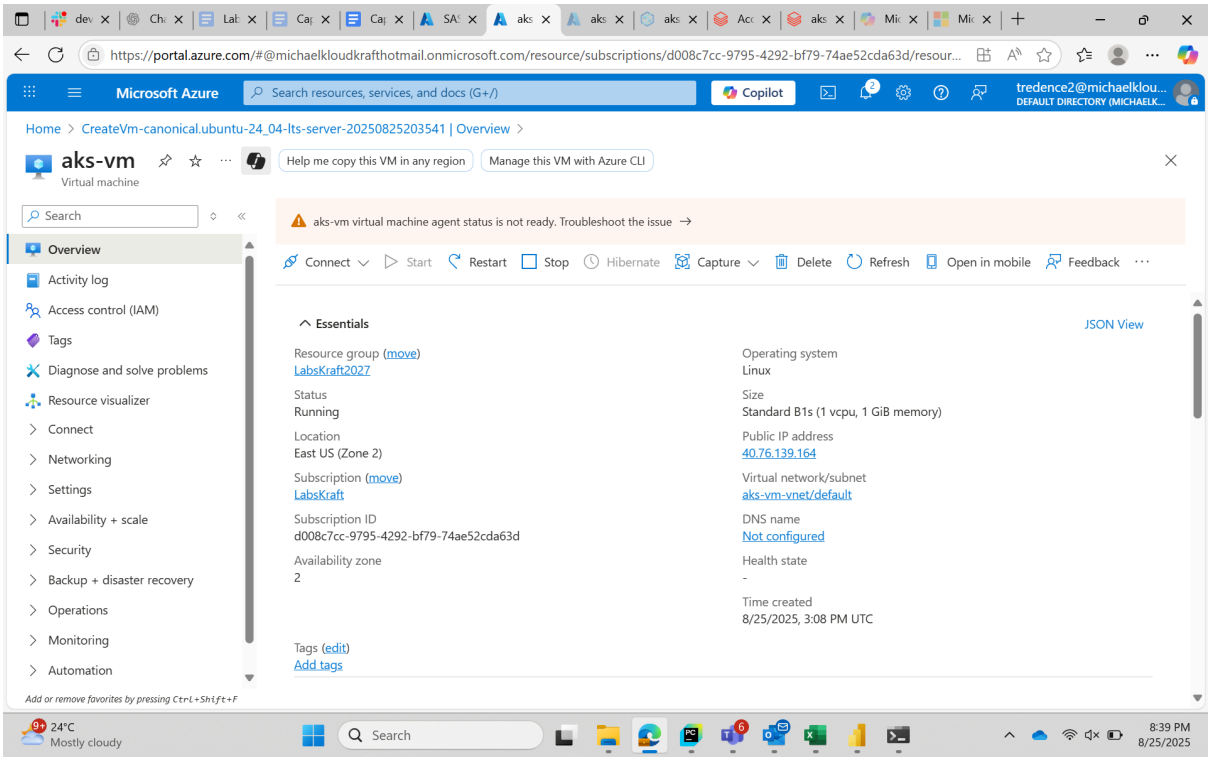
17) Creating namespace and event hub



18) Generating shared access policies



19) Creating a virtual machine to send data to the event hub



20) Generating data using the below python script to send data to the event hub

```
import random
 import time
 import uuid
 from datetime import datetime
 from azure.eventhub import EventHubProducerClient, EventData


# ----------------------------
 # CONFIGURATION
 # ----------------------------
 EVENT_HUB_CONNECTION_STR =
"Endpoint=sb://hetulproject01eventspace.servicebus.windows.net/;SharedAccessKeyName=
hello;SharedAccessKey=Cqw6MObIR0mIlhVGqOjbbEtH6s6lvgxI2+AEhDNW7r4=;EntityPath
=hetulhub"
 EVENT_HUB_NAME = "hetulhub"



# ----------------------------
 # Function to generate one IoT row
 # ----------------------------
```

```
 def generate_row():
        quantity = random.randint(1, 10)
        unit_price = round(random.uniform(50, 500), 2)
        sales_amount = round(quantity * unit_price, 2)
        device_status = random.choice(["Active", "Inactive", "Error"])
        anomaly_flag = 1 if device_status != "Active" else 0
        anomaly_type = random.choice(["Overheat","LowBattery","Disconnected"]) if
anomaly_flag else "None"



        return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North","South","East","West"]),
        "StoreID": random.choice(["Store001","Store002","Store003"]),
        "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
        }

# ----------------------------
 # Event Hub Producer
 # ----------------------------
 producer = EventHubProducerClient.from_connection_string(
        conn_str=EVENT_HUB_CONNECTION_STR,
        eventhub_name=EVENT_HUB_NAME
 )



# ----------------------------
 # Send data every second
 # ----------------------------
 try:
        for _ in range(1000):  # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row))  # convert dict to string
        with producer:
        producer.send_batch([event_data])
        print(f"Sent row: {row}")
```

```
        time.sleep(1)  # 1-second interval
except KeyboardInterrupt:
        print("Streaming stopped by user.")
finally:
        producer.close()
```

21) Setting up event hub config in databricks

```
# Event Hub connection parameters

eh_namespace = "aks-eventhub"

eh_name = "aksevent"

eh_sas_key_name = "akspolicy"

eh_sas_key = "PNxlad35DqJlmuKdR3b0SUl45WH7064Lo+AEhPthK/0="



connection_string =
f"Endpoint=sb://aks-eventhub.servicebus.windows.net/;SharedAccessKeyName=aksp
olicy;SharedAccessKey=PNxlad35DqJlmuKdR3b0SUl45WH7064Lo+AEhPthK/0=;EntityPath
=aksevent"
```

```
from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, TimestampType, IntegerType

from pyspark.sql import functions as F



# Define schema for Event Hub messages

event_schema = StructType([

    StructField("OrderID", StringType(), True),

    StructField("CustomerID", StringType(), True),

    StructField("Region", StringType(), True),

    StructField("StoreID", StringType(), True),
```

```
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),       # can cast to
Timestamp later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

# Read streaming data from Event Hub
stream_df = (spark.readStream
    .format("eventhubs")
    .option("eventhubs.connectionString", connection_string)
    .load()
)

# Event Hub messages are in 'body' column as binary, decode them
decoded_df = (stream_df
    .withColumn("body_str", F.col("body").cast("string"))
    .select(F.from_json("body_str", event_schema).alias("data"))
    .select("data.*")
```

```
)


# Write stream to Bronze Delta table
bronze_stream = (decoded_df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")
    .table("aks_proj1.bronze.events_bronze")  # new bronze table for streaming
)
```

We get an error because we don't have admin access.